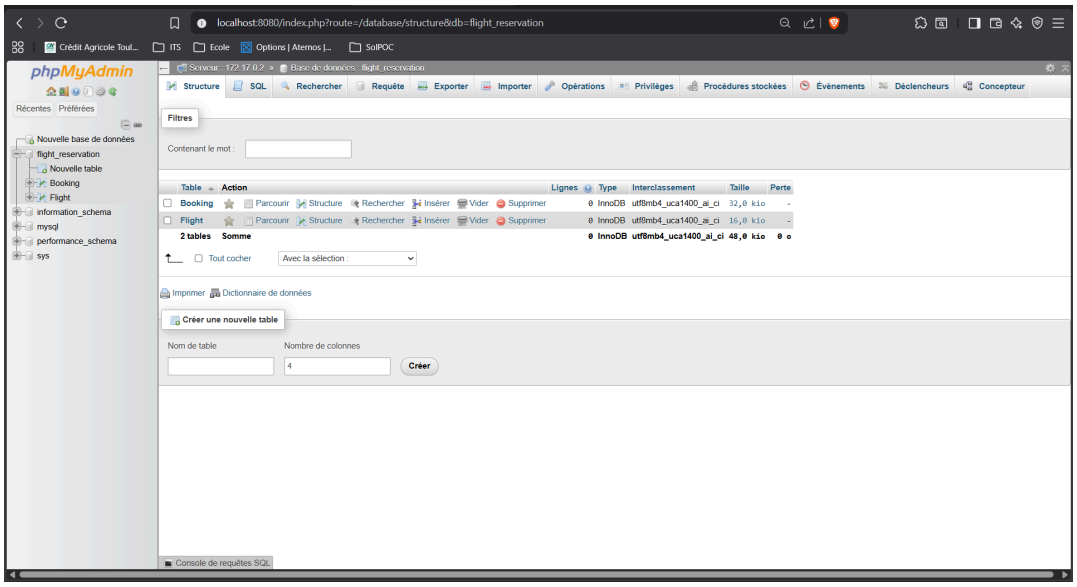
Second part: Use case “Flight reservation app”

Exercise 3: Create the database in MariaDB.

- **How is the “concern” relation translated in the Logical Data Model ? (Look at the cardinality)**

Dans un modèle logique de données, la relation « concern » est traduite selon sa cardinalité : si elle est en 1:N, on ajoute une clé étrangère dans la table du côté N ; si elle est en 1:1, on place une clé étrangère dans l’une des deux tables (souvent avec une contrainte d’unicité) ; et si elle est en N:N, on crée une table associative contenant les clés étrangères des deux entités.

- **Write the SQL CREATE TABLE instructions that will create those tables.**




```
CREATE TABLE Flight (
    id_flight VARCHAR(20) PRIMARY KEY,
    maximum_capacity INT NOT NULL CHECK (maximum_capacity <= 999),
    current_capacity INT NOT NULL CHECK (current_capacity <= 999)
);

CREATE TABLE Booking (
    id_booking INT AUTO_INCREMENT PRIMARY KEY,
    client_name VARCHAR(50) NOT NULL,
    quantity INT NOT NULL CHECK (quantity <= 99),
    id_flight VARCHAR(20) NOT NULL,
    FOREIGN KEY (id_flight) REFERENCES Flight(id_flight)
);
```

Verified by the teacher 🙂

Exercise 5: Adding new data to the database

Exercise 5.1: Add a flight.

```
cur = conn.cursor()

id_flight = "FL100"

cur.execute("SELECT * FROM Booking WHERE id_flight = ?", (id_flight,))

for row in cur:
    print(row)
```

[6] ✓ 0.0s

```
(1, 'Alice Dupont', 2, 'FL100')
(2, 'Bob Martin', 1, 'FL100')
```

```
print(cur._transformed_statement)
```

[7] ✓ 0.0s

```
b"SELECT * FROM Booking WHERE id_flight = 'FL100'"
```

Exercise 5.2: Add a booking.

```
addFlight("LH6795", 10)
print(addBooking("LH6795","Client 0",1))
print(addBooking("LH6795","Client 1",4))
print(addBooking("LH6795","Client 2",3))
print(addBooking("LH6795","Client 3",3))
```

✓ 12.1s

```
Flight ajouté avec succès
(False, 'Reserved')
(False, 'Reserved')
(False, 'Reserved')
(False, 'Full')
```

The results are consistent: the last booking is Fullbecause it exceeds capacity.

Exercise 6: Multi-threading

- **What do you see? What are the values in the Flight tables? How do you explain them? Which are the clients that were able to reserve their flight? (Compare to the sequential test of Ex. 5)**

```python
# Test parameters
id_flight="LH6795"
base_name="Client "
# Clean the db for the test
cleanDB(id_flight)
# Creating several Thread representing several client that try to booksimultaneously a random number of seat
jobs=[]
for i in range(5):
    jobs.append(threading.Thread(target=processBooking,args=(id_flight,base_name+str(i), random.randint(1,4))))

# Start each job, giving 3s of delay between each
for job in jobs:
    job.start()
    time.sleep(3)
# Wait for all the thread to finish
for job in jobs:
    job.join()
print("Finished")
```

✓ 16.1s

```
Processing Flight LH6795 for Client 0 : 2 seats.
Processing Flight LH6795 for Client 1 : 1 seats.
Client 0 : Reserved Should retry: False
Processing Flight LH6795 for Client 2 : 1 seats.
Error: Record has changed since last read in table 'Booking'
Client 1 : Error Should retry: True
Processing Flight LH6795 for Client 3 : 1 seats.
Client 2 : Reserved Should retry: False
Processing Flight LH6795 for Client 4 : 3 seats.
Error: Record has changed since last read in table 'Booking'
Client 3 : Error Should retry: True
Client 4 : Reserved Should retry: False
Finished
```

In concurrent execution, multiple threads access the same data at the same time, causing conflicts and errors. Unlike sequential execution, not all bookings succeed because the database prevents inconsistencies by rejecting some transactions, so only some clients reserve while others fail or must retry.

- **Comment the time.sleep(3) line and re-run the simulation. What do you see? How do you explain this message/behaviour?**

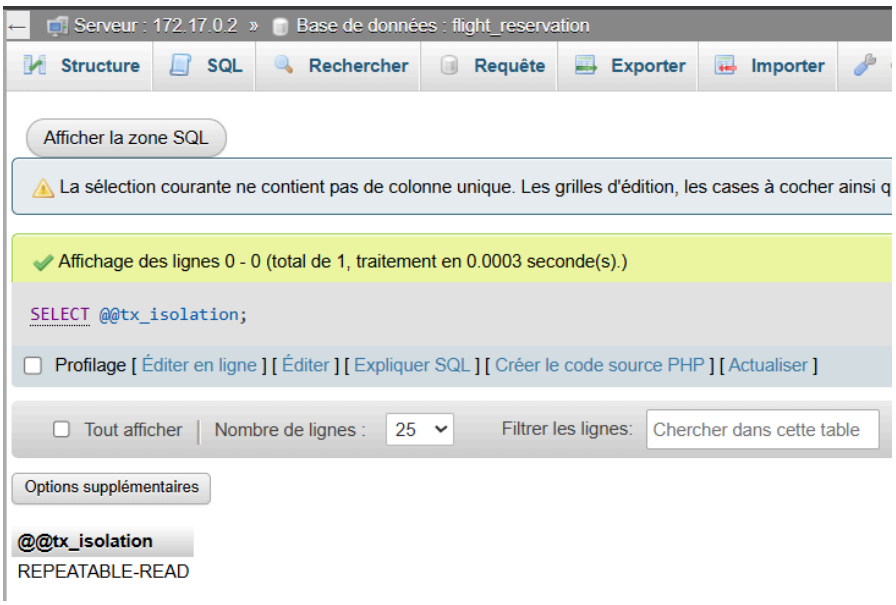
```python
# Test parameters
id_flight="LH6795"
base_name="Client "
# Clean the db for the test
cleanDB(id_flight)
# Creating several Thread representing several client that try to booksimultaneously a random number of seat
jobs=[]
for i in range(5):
    jobs.append(threading.Thread(target=processBooking,args=(id_flight,base_name+str(i), random.randint(1,4))))

# Start each job, giving 3s of delay between each
for job in jobs:
    job.start()
    #time.sleep(3)
# Wait for all the thread to finish
for job in jobs:
    job.join()
print("Finished")
```

✓ 5.1s

```
Processing FlightProcessing Flight LH6795 for Client 1 : 4 seats.
 LH6795 for Client 0 : 2 seats.
Processing Flight LH6795 for Client 2 : 3 seats.
Processing Flight LH6795 for Client 3 : 2 seats.
Processing Flight LH6795 for Client 4 : 4 seats.
Client 1 : Reserved Should retry: False
Error: Record has changed since last read in table 'Booking'
Client 2 : Error Should retry: True
Error: Record has changed since last read in table 'Booking'
Client 3 : Error Should retry: True
Error: Record has changed since last read in table 'Booking'
Client 0 : Error Should retry: True
Error: Record has changed since last read in table 'Booking'
Client 4 : Error Should retry: True
Finished
```

Without delay, concurrency is higher, leading to more conflicts and deadlocks, so most transactions fail and only a few succeed.

- **Run the SELECT @@tx_ISOLATION; query (in python or in phpMyAdmin) What is the current Isolation Level? How does it explain the observed behaviour?**



The isolation level is REPEATABLE READ. It means each transaction reads the same snapshot of the data, even if other transactions modify it. This leads to conflicts and deadlocks when multiple threads try to update the same row concurrently.

- **Modify your addBooking function by adding the following code just before the first SELECT. Then re-run the simulation. cur.execute("SET TRANSACTION ISOLATION LEVEL SERIALIZABLE")**
  What do you see? How do you explain this message/behaviour? How many clients were able to reserve their flight? Why?

```
Processing Flight LH6795 for Client 0 : 2 seats.
Processing Flight LH6795 for Client 1 : 4 seats.
Processing Flight LH6795 for Client 2 : 4 seats.
Processing Flight LH6795 for Client 3 : 4 seats.
Processing Flight LH6795 for Client 4 : 2 seats.
Error:Error: Deadlock found when trying to get lock; try restarting transaction
 Deadlock found when trying to get lock; try restarting transaction
Client 2 : Error Should retry: True
Client 4 : Error Should retry: True
Error: Deadlock found when trying to get lock; try restarting transaction
Error: Deadlock found when trying to get lock; try restarting transaction
Client 0 : Error Should retry: True
Client 1 : Error Should retry: True
Client 3 : Reserved Should retry: False
Finished
```

With SERIALIZABLE, strong locking causes many deadlocks, so only one transaction succeeds while others are rolled back to preserve consistency.

- **Modify your processBooking function by adding the following code after the print (watch for the indentation). Then re-run the simulation.**
  What do you see? How do you explain this message/behaviour? How many clients were able to reserve their flight? What is the final capacity of the flight? What can you say of the order of reservation?

```
 Deadlock found when trying to get lock; try restarting transaction
Client 2 : Error Should retry: True
Retry Flight LH6795 for Client 2 : 4 seats.
Client 0 : Error Should retry: True
Retry Flight LH6795 for Client 0 : 1 seats.
Client 3 : Error Should retry: True
Retry Flight LH6795 for Client 3 : 2 seats.
Client 4 : Reserved Should retry: False
Client 1 : Error Should retry: True
Retry Flight LH6795 for Client 1 : 3 seats.
Error:Error: Deadlock found when trying to get lock; try restarting transaction
 Deadlock found when trying to get lock; try restarting transaction
Error: Deadlock found when trying to get lock; try restarting transaction
Client 3 : Error
Retry Flight LH6795 for Client 3 : 2 seats.
Client 2 : Error
Retry Flight LH6795 for Client 2 : 4 seats.
Client 1 : Error
Retry Flight LH6795 for Client 1 : 3 seats.
Client 0 : Reserved
Error: Deadlock found when trying to get lock; try restarting transaction
Error: Deadlock found when trying to get lock; try restarting transaction
Client 1 : Error
Retry Flight LH6795 for Client 1 : 3 seats.
Client 2 : Reserved
Client 3 : Error
Retry Flight LH6795 for Client 3 : 2 seats.
Client 3 : Full
Client 1 : Full
Finished
```

With retries, deadlocks are handled by re-executing transactions, allowing more bookings to succeed. The final capacity is consistent, but the reservation order is not predictable due to concurrency and locking.

- **exo6.6 What do you see? How do you explain this message/behaviour?**

The error occurs because transactions are blocked by locks held by other concurrent transactions. With SERIALIZABLE isolation, locking is stronger, leading to timeouts when accessing the same data.

- **Write a general conclusion on transactions and isolation level. Why cannot we be always in SERIALIZABLE? In REPETABLE READ? Is there other isolation levels and why?**

Transactions ensure data consistency and integrity by grouping operations into atomic units, while isolation levels control how concurrent transactions interact. Higher isolation levels like SERIALIZABLE provide the strongest consistency by preventing all conflicts, but they cause many locks, deadlocks, and poor performance, so they are not always practical. REPEATABLE READ offers a balance but can still lead to conflicts due to concurrent updates. Lower levels like READ COMMITTED and READ UNCOMMITTED reduce locking and improve performance, but allow anomalies (e.g., dirty or non-repeatable reads). Therefore, different isolation levels exist to balance consistency and performance depending on the application needs.

Third part: Benchmarking of an existing database

- **Retrieve the SQL dump of the database and import it into a new database from PhpMyAdmin. What happens when you try to import it? How could you fix this issue?**

The import fails mainly due to file size and server limits. Compressing the file (ZIP) works because it reduces the upload size, but a more robust solution is to use the command line or increase server limits.

- **Respect the web application filename formalism and explore your database to get familiar with the values and the link between tables. Choose 3 queries you ran to do so and explain your choice.**

1- Understand the link between Booking and Flight :

```
SELECT b.booking_id, b.num_passengers, f.flight_day, f.flight_hour
FROM Booking b
JOIN Flight f ON b.booking_id = f.flight_id
LIMIT 10;
```

Affichage des lignes 0 - 9 (total de 10, traitement en 0.0006 seconde(s).)

```
SELECT b.booking_id, b.num_passengers, f.flight_day, f.flight_hour FROM Booking b JOIN Flight f ON b.booking_id = f.flight_id LIMIT 10;
```

Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

Options supplémentaires

booking_id	num_passengers	flight_day	flight_hour
1	1	Mon	4
2	2	Mon	9
3	1	Mon	14
4	2	Mon	19
5	1	Tue	4
6	2	Tue	14
7	1	Wed	15
8	3	Wed	17
9	2	Thu	6
10	1	Thu	14

2- Explore customer behavior (services requested) :

```
SELECT wants_extra_baggage, wants_preferred_seat, wants_in_flight_meals,
COUNT(*) as total
FROM Booking
GROUP BY wants_extra_baggage, wants_preferred_seat, wants_in_flight_meals;
```

Affichage des lignes 0 - 7 (total de 8, traitement en 0.0191 seconde(s).)

```
SELECT wants_extra_baggage, wants_preferred_seat, wants_in_flight_meals, COUNT(*) as total FROM Booking GROUP BY wants_extra_baggage, wants_preferred_seat, wants_in_flight_meals;
```

Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

Tout afficher | Nombre de lignes : 25 Filtrer les lignes: Chercher dans cette table

Options supplémentaires

wants_extra_baggage	wants_preferred_seat	wants_in_flight_meals	total
0	0	0	10456
0	0	1	3421
0	1	0	1552
0	1	1	1133
1	0	0	13243
1	0	1	8033
1	1	0	3393
1	1	1	8771

3- Analyze bookings by country and completion :

```
SELECT booking_origin, booking_complete, COUNT(*) as total
FROM Booking
GROUP BY booking_origin, booking_complete
ORDER BY total DESC;
```

Affichage des lignes 0 - 24 (total de 158, traitement en 0.0281 seconde(s).)

```
SELECT booking_origin, booking_complete, COUNT(*) as total FROM Booking GROUP BY booking_origin, booking_complete ORDER BY total DESC;
```

Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

1 > >> | Tout afficher | Nombre de lignes : 25 Filtrer les lignes: Chercher dans cette table

Options supplémentaires

booking_origin	booking_complete	total 1
Australia	0	16972
Malaysia	0	4706
South Korea	0	4097
Japan	0	3407
China	0	2693
Malaysia	1	2468
Taiwan	0	1853
Indonesia	0	1754
Thailand	0	1560
India	0	1139
New Zealand	0	1020
Australia	1	901
Singapore	0	745
China	1	694
Indonesia	1	615
Japan	1	478
Thailand	1	470

- **We will now perform resilience tests to evaluate if our database dump import is effective and repeatable without flaws. First, you will need to write a python script (in a .py format and NOT a notebook) to retrieve the dump and import it into our database as done earlier manually with PhpMyAdmin and improve it**

The import does not work properly because the database is not empty when the dump is executed.

- **8.2 You will now improve your import function by repeating the import several times (e.g. 5) and export a new dump (with a different name) each time to then compare them. What can you conclude about your dump resilience?**

```
--- Timing summary ---
Iteration 1 : import 2.65s | export 0.27s
Iteration 2 : import 2.78s | export 0.29s
Iteration 3 : import 2.70s | export 0.26s
Iteration 4 : import 2.56s | export 0.25s
Iteration 5 : import 2.73s | export 0.23s

--- Comparing dumps ---
dump_export_1.sql : 8855556 bytes
dump_export_2.sql : 8855556 bytes
dump_export_3.sql : 8855556 bytes
dump_export_4.sql : 8855556 bytes
dump_export_5.sql : 8855556 bytes
All dumps are identical in size -> RESILIENT
PS D:\OneDrive - Fondation EPF\Ubuntu\4A\S8\Data_Models\Lab 1 Robust DBMS for transactional>
```

All iterations produce dumps with identical sizes and similar execution times. This shows that the import/export process is consistent and reproducible. Therefore, the dump is resilient, as no variation or data inconsistency appears across multiple executions.

- **9.1 Write a query to check if 2 passengers of a same flight are assigned to the same seat.**

This query groups passengers by flight and seat. If more than one passenger is assigned to the same seat on the same flight, it highlights an inconsistency.

```sql
SELECT f.flight_id, r.seat, COUNT(*)
FROM Passenger p
JOIN Reserve r ON p.passenger_id = r.passenger_id
JOIN Booking b ON r.booking_id = b.booking_id
JOIN Flight f ON b.flight_id = f.flight_id
GROUP BY f.flight_id, r.seat
HAVING COUNT(*) > 1;
```

- **9.2 Write a query to check if a passenger booked 2 different flights overlapping each other in time (low probability but we might want to act if this happens so Murphy's law rules)**

```
SELECT p.passenger_id
FROM Passenger p
JOIN Reserve r1 ON p.passenger_id = r1.passenger_id
JOIN Booking b1 ON r1.booking_id = b1.booking_id
JOIN Flight f1 ON b1.flight_id = f1.flight_id

JOIN Reserve r2 ON p.passenger_id = r2.passenger_id
JOIN Booking b2 ON r2.booking_id = b2.booking_id
JOIN Flight f2 ON b2.flight_id = f2.flight_id

WHERE f1.flight_id < f2.flight_id
AND f1.flight_day = f2.flight_day
AND ABS(f1.flight_hour - f2.flight_hour) < 2;
```

This query uses a self-join to compare flights booked by the same passenger and detects overlapping schedules.

- **9.3 What other consistency test would you want to tackle periodically and why? Do it!**

This query checks if any passenger does not have a seat assigned. This situation should not happen in a valid booking, as each passenger must have a seat. Detecting such anomalies helps ensure data completeness and integrity.

```
SELECT p.passenger_id
FROM Passenger p
LEFT JOIN Reserve r ON p.passenger_id = r.passenger_id
WHERE r.seat IS NULL;
```

- **10. We will now measure the time spent running this series of tests to complete our benchmark with timestamps milestones between steps.**

```
--- Timing summary ---
Iteration 1 : import 2.88s | export 0.26s
Iteration 2 : import 2.71s | export 0.27s
Iteration 3 : import 2.94s | export 0.33s
Iteration 4 : import 2.83s | export 0.28s
Iteration 5 : import 2.75s | export 0.34s

--- Comparing dumps ---
dump_export_1.sql : 8855556 bytes
dump_export_2.sql : 8855556 bytes
dump_export_3.sql : 8855556 bytes
dump_export_4.sql : 8855556 bytes
dump_export_5.sql : 8855556 bytes
All dumps are identical in size -> RESILIENT

--- Consistency checks timing ---
same seat query : 0.27s
overlapping flights query : 0.38s
passenger without seat query : 0.21s
PS D:\OneDrive - Fondation EPF\Ubuntu\4A\S8\Data_Models\Lab 1 Robust DBMS for transactional>
```

The system is resilient and performant: dumps are identical across runs and consistency queries execute efficiently, ensuring reliable data integrity checks.

- **Finally, we will try to improve dump import time by using multi-threading. That is, splitting our SQL dump file into chunks that will be imported concurrently in separate threads. How quicker is this import method compared to before? Is it less reliable/resilient?**

PS D:\OneDrive - Fondation EPF\Ubuntu\4A\S8\Data_Models\Lab 1 Robust DBMS for transactional> python3 .\dump_import_threaded.py
MULTI-THREAD IMPORT (8 threads)
Splitting SQL file...

Import with 8 threads done in 0.47s

5 threads failed

Chunk 1 FAILED:

INSERT INTO `Booking` (`booking_id`, `num_passengers`, `sales_channel`, `trip_type`, `booking_origin`, `purchase_lead`, `booking_complete`, `wants_extra_baggage`, `wants_preferred_seat`, `wants_in_flight_meals`, `length_of_stay`, `flight_id`) VALUES
(17678, 1, 'Internet', 'RoundTrip',

Chunk 4 FAILED:

INSERT INTO `Passenger` (`passenger_id`, `lastname`, `firstname`, `email`) VALUES

```
(7179, 'Murray', 'Laura', 'Laura.Murray42@williams-anderson.org'),
(7181, 'Fields', 'Morgan', 'Morgan.Fields24@buchanan.biz'),
(7182, 'Mccullough', 'Ryan', 'Ryan.Mccullough12@rivera.com'),
(7183, 'Willia

Chunk 5 FAILED:
--------------
INSERT INTO `Passenger` (`passenger_id`, `lastname`, `firstname`, `email`) VALUES
(35384, 'Cordova', 'Cody', 'Cody.Cordova39@hancock-smith.net'),
(35386, 'Trujillo', 'Paul', 'Paul.Trujillo44@brown-graves.org'),
(35387, 'Scott', 'Brittany', 'Brittany.Scott72@malone-thomas.net'),
(35388

Chunk 6 FAILED:
--------------
INSERT INTO `Passenger` (`passenger_id`, `lastname`, `firstname`, `email`) VALUES
(69865, 'Watts', 'Riley', 'Riley.Watts82@brown.com'),
(69868, 'Potter', 'Travis', 'Travis.Potter91@bennett.com'),
(69869, 'Gaines', 'John', 'John.Gaines99@sanders.com'),
(69871, 'Blair', 'Steven', 'Steve

Chunk 7 FAILED:
--------------
INSERT INTO `Reserve` (`booking_id`, `passenger_id`, `seat`) VALUES
(32475, 25537, '18B'),
(32476, 229, '16C'),
(32477, 47635, '7D'),
(32478, 24032, '6D'),
(32478, 47636, '7A'),
(32478, 47637, '5D'),
(32478, 47638, '6A'),
(32478, 47639, '19B'),
(32478, 47640, '19A'),
(32478, 47642, '1
```

The multi-threaded import significantly improves performance. In our experiment, the import time decreased from approximately **2.7 seconds to 0.46** seconds using 8 threads, resulting in a speedup of about 5–6 times.

However, this method is not reliable. Several threads failed during execution due to SQL dependencies and ordering constraints. Errors occurred when inserting data into tables before their dependencies were satisfied (e.g., foreign key constraints or missing tables). This leads to incomplete or inconsistent database states.

While multi-threading greatly improves performance, it breaks the sequential nature required by SQL dumps. Therefore, it is faster but significantly less resilient and should not be used without careful handling of dependencies.